```
!pip install torchinfo --quiet
```

```
import time
import math
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader, random_split

SEED = 42
torch.manual_seed(SEED)
np.random.seed(SEED)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)


def count_parameters(model):
    return sum(p.numel() for p in model.parameters() if p.requires_gra


def model_size_mb(model):
    return count_parameters(model) * 4 / (1024 ** 2)


def estimate_recurrent_flops(cell_type, input_size, hidden_size, seq_
                             num_layers=1, output_size=None):
    """Analytical forward FLOPs per sequence for comparing cell types.
    Gate multipliers: RNN=1, GRU=3, LSTM=4."""
    gate_mult = {"rnn": 1, "gru": 3, "lstm": 4}[cell_type.lower()]
    flops = 0
    in_sz = input_size
    for layer in range(num_layers):
        per_step = gate_mult * 2 * (in_sz * hidden_size + hidden_size
        flops += per_step * seq_len
        in_sz = hidden_size
    if output_size is not None:
        flops += 2 * hidden_size * output_size
    return flops
```

```
Using device: cuda
```

```
text_p1 = ("Next character prediction is a fundamental task in the fiel
"processing (NLP) that involves predicting the next character in a sequ
"the characters that precede it. This task is essential for various app
"text auto-completion, spell checking, and even in the development of s
"capable of generating human-like text. "
"At its core, next character prediction relies on statistical models or
"algorithms to analyze a given sequence of text and predict which chara
"follow. These predictions are based on patterns and relationships lear
"of text during the training phase of the model. "
"One of the most popular approaches to next character prediction involv
"Neural Networks (RNNs), a variant called Long S
"networks. RNNs are particularly well-suited for sequential data like t
"maintain information in 'memory' about previous characters to inform t
"next character. LSTM networks enhance this capability by being able to
"dependencies, making them even more effective for next character predi
"Training a model for next character prediction involves feeding it lar
"data, allowing it to learn the probability of each character's appeara
"sequence of characters. During this training process, the model adjust
"minimize the difference between its predictions and the actual outcome
"predictive accuracy over time. "
"Once trained, the model can be used to predict the next character in a
"considering the sequence of characters that precede it. This can enhan
"text editing software, improve efficiency in coding environments with
"features, and enable more natural interactions with AI-based chatbots
"In summary, next character prediction plays a crucial role in enhancin
"various NLP applications, making text-based interactions more efficien
"human-like. Through the use of advanced machine learning models like R
"character prediction continues to evolve, opening new possibilities fo
"text-based technology.")

print("Corpus length (chars):", len(text_p1))
chars_p1 = sorted(list(set(text_p1)))
vocab_p1 = len(chars_p1)
print("Vocabulary size:", vocab_p1)

char_to_ix_p1 = {ch: i for i, ch in enumerate(chars_p1)}
ix_to_char_p1 = {i: ch for i, ch in enumerate(chars_p1)}
```

```
Corpus length (chars): 2386
Vocabulary size: 44
```

```
def make_dataset(text, char_to_ix, seq_length):
    X, y = [], []
    for i in range(len(text) - seq_length):
```

```
        seq = text[i:i + seq_length]
        label = text[i + seq_length]
        X.append([char_to_ix[c] for c in seq])
        y.append(char_to_ix[label])
    X = torch.tensor(X, dtype=torch.long)
    y = torch.tensor(y, dtype=torch.long)
    return X, y


class CharSequenceModel(nn.Module):
    """Embedding -> recurrent (RNN/LSTM/GRU) -> linear head on last tim
    def __init__(self, vocab_size, embed_size, hidden_size, output_size
                 cell_type="rnn", num_layers=1):
        super().__init__()
        self.cell_type = cell_type.lower()
        self.num_layers = num_layers
        self.hidden_size = hidden_size
        self.embedding = nn.Embedding(vocab_size, embed_size)
        rnn_cls = {"rnn": nn.RNN, "lstm": nn.LSTM, "gru": nn.GRU}[self.
        self.rnn = rnn_cls(embed_size, hidden_size, num_layers=num_laye
                           batch_first=True)
        self.fc = nn.Linear(hidden_size, output_size)

    def forward(self, x):
        embedded = self.embedding(x)
        output, _ = self.rnn(embedded)
        return self.fc(output[:, -1, :])


def train_eval(model, X_train, y_train, X_val, y_val, epochs=50, lr=0.0
               batch_size=128, log_every=10, clip_norm=5.0, label=""):
    model = model.to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=lr)

    X_train, y_train = X_train.to(device), y_train.to(device)
    X_val, y_val = X_val.to(device), y_val.to(device)

    n = X_train.size(0)
    train_loss_hist, val_loss_hist, val_acc_hist = [], [], []

    start = time.time()
    for epoch in range(epochs):
        model.train()
        perm = torch.randperm(n)
        epoch_loss = 0.0
```

```
        epoch_loss = 0.0
        for i in range(0, n, batch_size):
            idx = perm[i:i + batch_size]
            xb, yb = X_train[idx], y_train[idx]
            optimizer.zero_grad()
            out = model(xb)
            loss = criterion(out, yb)
            loss.backward()
            torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm
            optimizer.step()
            epoch_loss += loss.item() * xb.size(0)
        epoch_loss /= n

        model.eval()
        with torch.no_grad():
            val_out = model(X_val)
            val_loss = criterion(val_out, y_val).item()
            pred = val_out.argmax(1)
            val_acc = (pred == y_val).float().mean().item()

        train_loss_hist.append(epoch_loss)
        val_loss_hist.append(val_loss)
        val_acc_hist.append(val_acc)

        if (epoch + 1) % log_every == 0:
            print(f"[{label}] Epoch {epoch+1}/{epochs}  "
                  f"train_loss={epoch_loss:.4f}  val_loss={val_loss:.4f
                  f"val_acc={val_acc:.4f}")

    train_time = time.time() - start
    return {
        "train_loss": train_loss_hist,
        "val_loss": val_loss_hist,
        "val_acc": val_acc_hist,
        "final_train_loss": train_loss_hist[-1],
        "final_val_loss": val_loss_hist[-1],
        "final_val_acc": val_acc_hist[-1],
        "best_val_acc": max(val_acc_hist),
        "best_val_epoch": int(np.argmax(val_acc_hist)) + 1,
        "train_time_s": train_time,
    }
```

```
from sklearn.model_selection import train_test_split

P1_EMBED = 64
```

```
P1_HIDDEN = 128
P1_EPOCHS = 50
P1_LR = 0.005
seq_lengths_p1 = [10, 20, 30]
cell_types = ["rnn", "lstm", "gru"]

p1_results = {}
p1_rows = []

for seq_len in seq_lengths_p1:
    X, y = make_dataset(text_p1, char_to_ix_p1, seq_len)
    X_tr, X_val, y_tr, y_val = train_test_split(
        X, y, test_size=0.2, random_state=SEED)
    for cell in cell_types:
        torch.manual_seed(SEED)
        model = CharSequenceModel(vocab_p1, P1_EMBED, P1_HIDDEN, vocab_
                                  cell_type=cell)
        label = f"{cell.upper()}-L{seq_len}"
        hist = train_eval(model, X_tr, y_tr, X_val, y_val,
                          epochs=P1_EPOCHS, lr=P1_LR, label=label)
        flops = estimate_recurrent_flops(cell, P1_EMBED, P1_HIDDEN, seq
                                         num_layers=1, output_size=voca
        p1_results[(cell, seq_len)] = hist
        p1_rows.append({
            "Model": cell.upper(),
            "Seq Len": seq_len,
            "Final Train Loss": round(hist["final_train_loss"], 4),
            "Final Val Loss": round(hist["final_val_loss"], 4),
            "Final Val Acc": round(hist["final_val_acc"], 4),
            "Best Val Acc": round(hist["best_val_acc"], 4),
            "Best Epoch": hist["best_val_epoch"],
            "Train Time (s)": round(hist["train_time_s"], 2),
            "Params": count_parameters(model),
            "Size (MB)": round(model_size_mb(model), 3),
            "FLOPs/seq": flops,
        })

p1_df = pd.DataFrame(p1_rows)
p1_df
```

```
[RNN-L10] Epoch 10/50  train_loss=0.9561  val_loss=1.9847  val_acc=0.51
[RNN-L10] Epoch 20/50  train_loss=0.2700  val_loss=2.4688  val_acc=0.47
[RNN-L10] Epoch 30/50  train_loss=0.1027  val_loss=2.7370  val_acc=0.49
[RNN-L10] Epoch 40/50  train_loss=0.0605  val_loss=3.0019  val_acc=0.47
[RNN-L10] Epoch 50/50  train_loss=0.0534  val_loss=3.1345  val_acc=0.48
[LSTM-L10] Epoch 10/50  train_loss=1.2080  val_loss=1.8991  val_acc=0.5
```

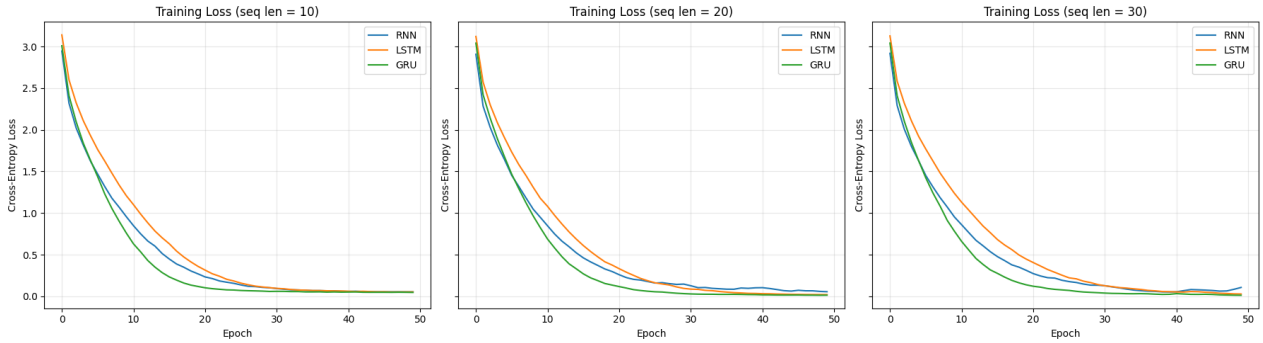
```
[LSTM-L10] Epoch 10/50  train_loss=1.2060  val_loss=1.8991  val_acc=0.5
[LSTM-L10] Epoch 20/50  train_loss=0.3603  val_loss=2.1749  val_acc=0.5
[LSTM-L10] Epoch 30/50  train_loss=0.1040  val_loss=2.5095  val_acc=0.5
[LSTM-L10] Epoch 40/50  train_loss=0.0641  val_loss=2.6579  val_acc=0.5
[LSTM-L10] Epoch 50/50  train_loss=0.0533  val_loss=2.7802  val_acc=0.4
[GRU-L10] Epoch 10/50  train_loss=0.7595  val_loss=1.9218  val_acc=0.55
[GRU-L10] Epoch 20/50  train_loss=0.1186  val_loss=2.4171  val_acc=0.50
[GRU-L10] Epoch 30/50  train_loss=0.0588  val_loss=2.6364  val_acc=0.51
[GRU-L10] Epoch 40/50  train_loss=0.0506  val_loss=2.7546  val_acc=0.51
[GRU-L10] Epoch 50/50  train_loss=0.0472  val_loss=2.8485  val_acc=0.51
[RNN-L20] Epoch 10/50  train_loss=0.9473  val_loss=2.0330  val_acc=0.48
[RNN-L20] Epoch 20/50  train_loss=0.2984  val_loss=2.4093  val_acc=0.49
[RNN-L20] Epoch 30/50  train_loss=0.1469  val_loss=2.8121  val_acc=0.46
[RNN-L20] Epoch 40/50  train_loss=0.1026  val_loss=3.1208  val_acc=0.48
[RNN-L20] Epoch 50/50  train_loss=0.0557  val_loss=3.3255  val_acc=0.47
[LSTM-L20] Epoch 10/50  train_loss=1.1760  val_loss=1.8490  val_acc=0.5
[LSTM-L20] Epoch 20/50  train_loss=0.3761  val_loss=2.0232  val_acc=0.5
[LSTM-L20] Epoch 30/50  train_loss=0.0958  val_loss=2.3734  val_acc=0.5
[LSTM-L20] Epoch 40/50  train_loss=0.0349  val_loss=2.5047  val_acc=0.5
[LSTM-L20] Epoch 50/50  train_loss=0.0208  val_loss=2.5946  val_acc=0.5
[GRU-L20] Epoch 10/50  train_loss=0.8229  val_loss=1.8438  val_acc=0.54
[GRU-L20] Epoch 20/50  train_loss=0.1354  val_loss=2.2585  val_acc=0.51
[GRU-L20] Epoch 30/50  train_loss=0.0328  val_loss=2.5907  val_acc=0.50
[GRU-L20] Epoch 40/50  train_loss=0.0201  val_loss=2.7197  val_acc=0.51
[GRU-L20] Epoch 50/50  train_loss=0.0143  val_loss=2.8017  val_acc=0.51
[RNN-L30] Epoch 10/50  train_loss=0.9524  val_loss=2.0803  val_acc=0.47
[RNN-L30] Epoch 20/50  train_loss=0.3127  val_loss=2.4872  val_acc=0.49
[RNN-L30] Epoch 30/50  train_loss=0.1336  val_loss=2.9099  val_acc=0.47
[RNN-L30] Epoch 40/50  train_loss=0.0521  val_loss=3.1188  val_acc=0.47
[RNN-L30] Epoch 50/50  train_loss=0.1068  val_loss=3.3547  val_acc=0.48
[LSTM-L30] Epoch 10/50  train_loss=1.2362  val_loss=1.9562  val_acc=0.5
[LSTM-L30] Epoch 20/50  train_loss=0.4480  val_loss=2.1542  val_acc=0.5
[LSTM-L30] Epoch 30/50  train_loss=0.1416  val_loss=2.5316  val_acc=0.4
[LSTM-L30] Epoch 40/50  train_loss=0.0557  val_loss=2.7687  val_acc=0.4
[LSTM-L30] Epoch 50/50  train_loss=0.0284  val_loss=2.9230  val_acc=0.4
[GRU-L30] Epoch 10/50  train_loss=0.7810  val_loss=2.0381  val_acc=0.51
[GRU-L30] Epoch 20/50  train_loss=0.1385  val_loss=2.5263  val_acc=0.51
[GRU-L30] Epoch 30/50  train_loss=0.0436  val_loss=2.8933  val_acc=0.50
[GRU-L30] Epoch 40/50  train_loss=0.0246  val_loss=3.0864  val_acc=0.48
[GRU-L30] Epoch 50/50  train_loss=0.0131  val_loss=3.2677  val_acc=0.49
```

	Model	Seq Len	Final Train Loss	Final Val Loss	Final Val Acc	Best Val Acc	Best Epoch	Train Time (s)	Params	Size (MB)	FL
0	RNN	10	0.0534	3.1345	0.4811	0.5168	10	2.65	33324	0.127	
1	LSTM	10	0.0533	2.7802	0.4937	0.5378	16	1.99	107820	0.411	
2	GRU	10	0.0472	2.8485	0.5168	0.5546	10	2.01	82988	0.317	
3	RNN	20	0.0557	3.3255	0.4705	0.5148	13	2.02	33324	0.127	

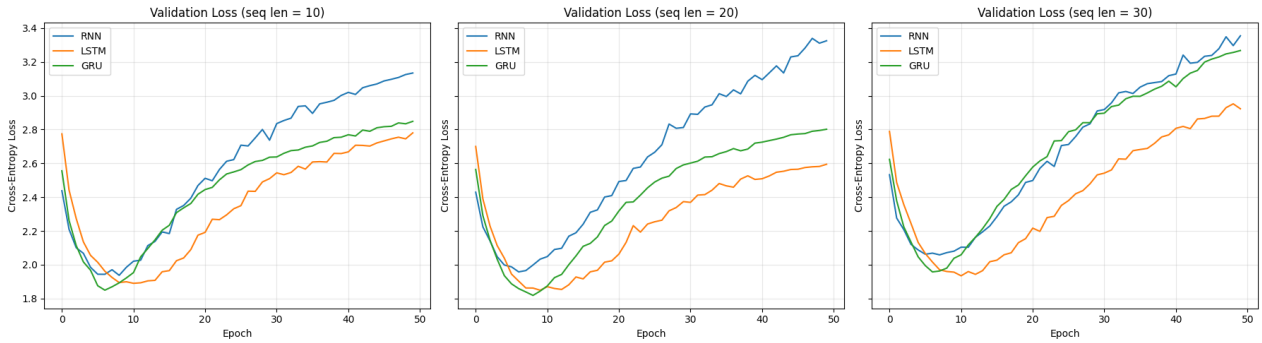
4	LSTM	20	0.0208	2.5946	0.5211	0.5359	17	2.00	107820	0.411
5	GRU	20	0.0143	2.8017	0.5169	0.5506	11	2.23	82988	0.317
6	RNN	30	0.1068	3.3547	0.4831	0.5127	24	1.97	33324	0.127
7	LSTM	30	0.0284	2.9230	0.4936	0.5254	16	2.14	107820	0.411

Next steps: Generate code with p1_df New interactive sheet

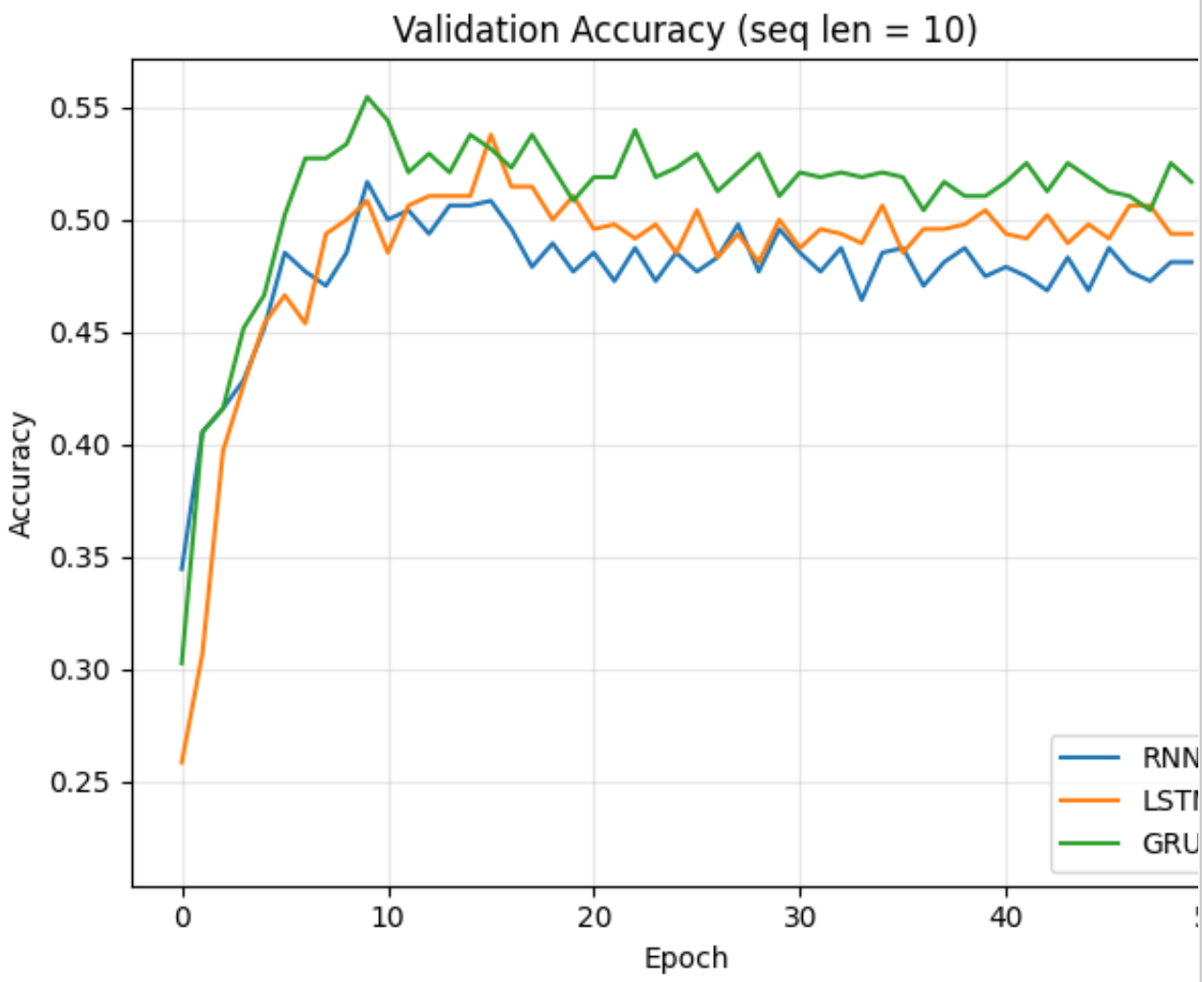
```
fig, axes = plt.subplots(1, len(seq_lengths_p1), figsize=(18, 5), sha
for ax, seq_len in zip(axes, seq_lengths_p1):
    for cell in cell_types:
        ax.plot(p1_results[(cell, seq_len)]["train_loss"], label=cell
    ax.set_title(f"Training Loss (seq len = {seq_len})")
    ax.set_xlabel("Epoch")
    ax.set_ylabel("Cross-Entropy Loss")
    ax.legend()
    ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



```
fig, axes = plt.subplots(1, len(seq_lengths_p1), figsize=(18, 5), sha
for ax, seq_len in zip(axes, seq_lengths_p1):
    for cell in cell_types:
        ax.plot(p1_results[(cell, seq_len)]["val_loss"], label=cell.u
    ax.set_title(f"Validation Loss (seq len = {seq_len})")
    ax.set_xlabel("Epoch")
    ax.set_ylabel("Cross-Entropy Loss")
    ax.legend()
    ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



```
fig, axes = plt.subplots(1, len(seq_lengths_p1), figsize=(18, 5), sha
for ax, seq_len in zip(axes, seq_lengths_p1):
    for cell in cell_types:
        ax.plot(p1_results[(cell, seq_len)]["val_acc"], label=cell.up
    ax.set_title(f"Validation Accuracy (seq len = {seq_len})")
    ax.set_xlabel("Epoch")
    ax.set_ylabel("Accuracy")
    ax.legend()
    ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



```
fig, axes = plt.subplots(1, 2, figsize=(15, 5))
width = 0.25
x = np.arange(len(seq_lengths_p1))
for k, cell in enumerate(cell_types):
    accs = [p1_results[(cell, s)]["best_val_acc"] for s in seq_lengths
    times = [p1_results[(cell, s)]["train_time_s"] for s in seq_lengtl
    axes[0].bar(x + k*width, accs, width, label=cell.upper())
    axes[1].bar(x + k*width, times, width, label=cell.upper())
axes[0].set_title("Best Validation Accuracy")
axes[0].set_xticks(x + width); axes[0].set_xticklabels(seq_lengths_p1
axes[0].set_xlabel("Sequence Length"); axes[0].set_ylabel("Accuracy")
axes[1].set_title("Training Time")
axes[1].set_xticks(x + width); axes[1].set_xticklabels(seq_lengths_p1
axes[1].set_xlabel("Sequence Length"); axes[1].set_ylabel("Seconds");
plt.tight_layout()
plt.show()
```